



COS110 Extra Notes

Program Design: Introduction (University of Pretoria)



Scan to open on Studocu

COS110 2013

Some things you should know that are not in the textbook

Inline member functions

When the body of a member function is written inside the class declaration, then it is called an inline member function. Sometimes it is convenient to do this, especially when the body of a member function is small. For example:

```
class Thing {
    private:
        string name;
    public:
        void getName() const { return name; } // this is an inline function
        void someOtherFunction();           // this function is not inline
};
```

Only simple functions can be compiled as inline functions. If you write a function inline, the compiler can ignore your request for it to be inline if it is not possible or practical. Inline functions are compiled differently from non-inline functions. In the case of non-inline function calls the execution of the program jumps to some other place in memory where the compiled function is located. This involves some housekeeping overhead, because the program has to keep track on the stack of a number of things including the address of the instruction to return to when the function is complete.

With inline functions, the compiler puts the actual code of the function at the place where the function is called (called inline expansion). This means that the function is no longer “called” as with regular function calls. This can result in an improvement in performance, because there is no overhead associated with the function call. The downside of inline functions is that the same code is repeated with each function call in the compiled code, which means that the executable program can become larger, which in turn can lead to a decrease in performance.

To summarize, although inline functions are sometimes convenient for the programmer and can result in more efficient code, there are some disadvantages. One disadvantage is that the executable program can become larger and so become more inefficient. Another disadvantage is that it results in the class definition not being well separated from the class implementation. This is not good in terms of program design. For these reasons, only short (typically one-line) member functions should be declared inline.

Initialisation lists

In constructors, an initialisation list can be used to give initial values to data members before the actual statement of the function body are executed. For example:

```
class Thing {
    private:
        int value;
        string name;
    public:
        Thing() : value(0), name("default") {}
        Thing(int _value, string _name) : value(_value), name(_name) {}
};
```

The above code has the same effect (although with complex parameter objects, the above can be more efficient) as:

```
class Thing {
    private:
        int value;
        string name;
    public:
        Thing() {
            value = 0;
            name = "default";
        }
        Thing(int _value, string _name) {
            value = _value;
            name = _name;
        }
};
```

Note that if a class has a base class, then the base class constructor must be called first in the initialization list before any variables are initialised, for example, say `SuperThing` is the base class of `Thing`:

```
Thing(double _st, int _value, string _name) : SuperThing(_st), value(_value),
                                             name(_name) {}
```

Concept of the stack and the heap

The stack and the heap are both sections in memory. The stack is where data such as local variables, values of arguments for functions, return values from functions and function return addresses are stored. When variables go out of scope, the memory is automatically freed.

The heap is where dynamic memory is stored. Whenever you use the keyword `new`, memory is set aside on the heap and the `new` operator will return the memory address of the newly allocated memory, which you can then store in a local variable pointer on the stack. Memory that is allocated on the heap is not automatically freed. It is the programmer's responsibility to free up all data that has been allocated using `new`, by using the `delete` operator.

Safe coding when using pointers

1. When `new` is called, before you continue with anything, first check if memory has been successfully allocated. In most modern compilers, the program will throw an exception when memory could not be allocated. You will learn in Chapter 14 how to handle exceptions, but in the meantime (to show that you are thinking about all possibilities) you should check if the null pointer was returned by `new`. If so, terminate the program.
2. Before deleting memory, first check if that pointer is not null. You should only use `delete` on pointers that are not null. If the pointer is null, then do nothing.
3. After deleting memory set the pointer to null to prevent dangling references.
4. In copy constructors, and whenever deep copies need to be made, first check if the source that needed to be copied actually does refer to some memory. If not, don't allocate memory for the destination, but set the pointer to null.

For example, say you have a class called `Thing` that has a pointer data member called `linkedStuff` that refers to a dynamic array. The destructor and copy constructor below illustrate the above 4 points:

```
Thing::~~Thing() {
    if(linkedStuff) {                // point 2 above
        delete [] linkedStuff;
        linkedStuff = NULL;          // point 3 above
    }
}

Thing::Thing(const Thing& otherThing) {
    // code to initialise all the non-dynamic data members ...

    // now do the deep copying of the dynamic data members:
    if(otherThing.linkedStuff) {     // point 4 above
        linkedStuff = new ...        // allocate whatever data is needed
        if (!linkedStuff)             // point 1 above
            exit(EXIT_FAILURE);       // cannot continue without memory
        // code to copy the contents from otherThing.linkedStuff to linkedStuff
    }
    else
        linkedStuff = NULL;          // point 4 above
}
```
